

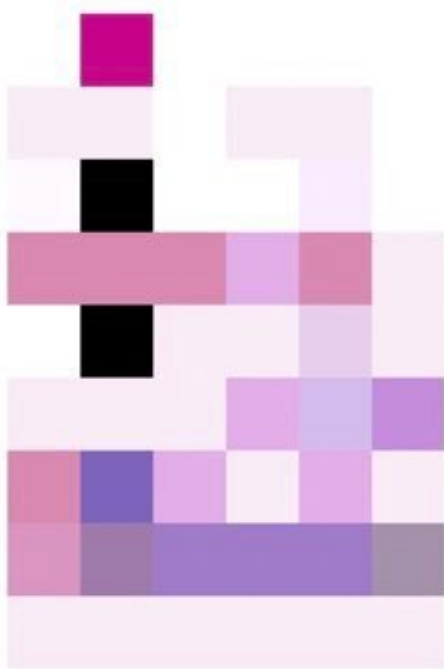
3>SHAKE

AIで実装は速くなった。
なのにプロダクトは速くならない。

職能の壁を越えて価値のフローを設計する

2026/09/05 Product Engineering Conference 2026

@nwiizo 30min



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

あらゆるサービスを連携するハブになる

クラウド型ETL/データパイプラインサービスの決定版

Security



セキュリファイ

事業者が抱えるセキュリティリスクを無くす

ワンストップで実現するセキュリティ対策

HR (Engineer Hiring)



リランス

良いエンジニアに良い案件を

フリーランスエンジニアに「今よりいい条件」を



この発表で解決できること

こんな状況ではありませんか

AIでPull Request（PR）は増え、手を動かす速さも上がったように感じる。けれど、価値が届くまでの時間や成果は、同じようには変わっていない。**速くなった実感**と、**届いた価値が噛み合わない**。

持ち帰れるもの

なぜ、実装が速くなってもプロダクト全体は速くならないのか。価値が届くまでの流れを測り、**全体の速さを決める詰まりを見つける手順**と、ユーザーが達成したいことから「作るべきか」「いつやめるか」を決める**判断の基準**を持ち帰れます。

見る範囲を、実装からユーザーの結果まで広げる。すると、次に何を速くすべきかが見えてくる。



本日の流れ

1. 増えた生産量はどこに消えたのか
2. 測りやすい数字は、価値が届いた証拠ではない
3. 役割ごとの前提を対話で確かめる
4. ユーザーが達成したいことから「作る・やめる」を判断する

前半は「なぜ速くならないのか」を数字で見ます。後半は、役割ごとの前提を確かめる**対話と一つの図**、チームの判断をAIの実装へ反映する**判断の前提・実装前の仕様**を扱います。



1. 増えた生産量はどこに消えたのか

個人は速くなったと感じる。組織の流れは別に測る。

AIで実装は確かに速くなった

AIを使うと、動くコードを書くまでの時間は短くなります。ただし、**個人が感じる作業の速さと、ユーザーや事業に起きた変化は、同じ指標では測れません。**

個人は速くなったと感じる

DORAは、ソフトウェア開発組織を継続的に調査しています。2025年の調査では、世界の技術職約5,000人のうち90%が仕事でAIを使い、80%以上が生産性の向上を感じている。

個人の体感だけでは組織の流れは決まらない

同じ調査では、AIをよく使う組織ほど、一定期間にユーザーへ届けた変更の件数が多く、ユーザーや事業の成果も高い傾向がありました。その一方で、変更による障害や手戻りは増えていました。DORAの2026年レポートも、コーディングが速くなるだけでは投資対効果（ROI）につながらないとしています。

AIを入れると、組織の強みは伸び、弱いところの問題も大きくなる。個人の体感と、ユーザーが結果を得るまでの流れは、別の指標で見る。

ジョブ理論では、ユーザーの目的から考える



クレイトン・M・クリステンセンほか『ジョブ理論』

Jobs to Be Done（ジョブ理論）は、ユーザーがなぜ製品やサービスを選ぶのかを、達成したいことから考える方法です。ここでいう**ジョブ**は、ユーザーがある状況で片付けたいことです。

状況 候補が多く、比較に時間がかかっている

ジョブ 必要な情報へ早くたどり着き、購入を判断したい

選べる手段 検索条件、比較表、担当者への相談

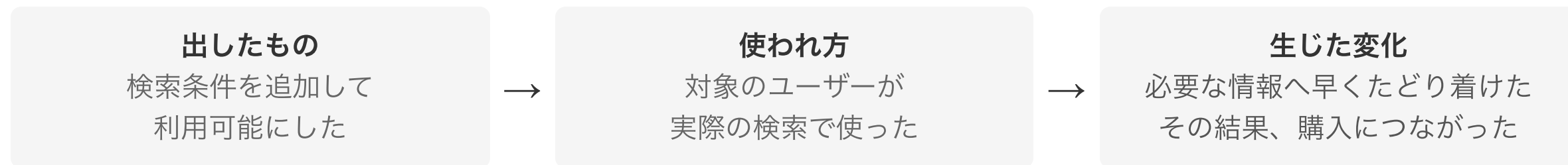
ジョブ理論では、ユーザーはジョブを片付けるためにプロダクトを「雇う」と表現します。検索機能は手段の一つです。別の手段の方がうまく片付くなら、ユーザーはそちらを選びます。

ジョブは機能名ではない。ユーザーが置かれた状況と、達成したいことを表す。

出典：Christensen Institute, Jobs to Be Done Theory / 書影：クレイトン・M・クリステンセンほか『ジョブ理論』

価値が届くのは、ユーザーのジョブが片付いたとき

機能を出した時点と、ユーザーがその機能を使ってジョブを片付けた時点を分けて考えます。検索機能を改善する例で見ると、次のようになります。



アウトプットは作って出したもの。**アウトカム**は、ユーザーが使ったあとに起きた、ユーザーや事業の変化です。リリースしただけでは、価値が届いたとは判断できません。利用後の変化を確かめて判断します。

プロダクトの速さは、機能を出すまでではなく、結果を確かめるまでで見る必要があります。

参考：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第1章、第11章 / Susanne Kaiser, *Architecture for Flow*, 第1章

たとえば、実作業が30%の流れなら

まず、チケットに着手してから変更を利用できるようになるまでを、実作業30%、待ち70%と置いた例で考えます。

実作業 30%

設計 ・ 実装 ・ テスト

待ち 70%

判断待ち ・ レビュー待ち ・ 他チーム待ち

この例では、実作業以外の7割が、仕様や範囲が決まるのを待つ時間、レビューと承認を待つ時間、依存先のチームの変更やリリースを待つ時間です。

比率は説明のための仮定です。自分たちの比率は、実際の記録から測ります。

実作業を半分にしても、全体は15%しか縮まない

いま



AIで実作業を半分にした後



全体を100と置くと、待ちの70は、実作業だけをAIで速めても残ります。実作業を半分にしても、全体の短縮は15%にとどまります。

この例では、合意と判断の待ち時間が、全体の速さを決める詰まりになる。

バリューストリームは発見から結果の確認まで

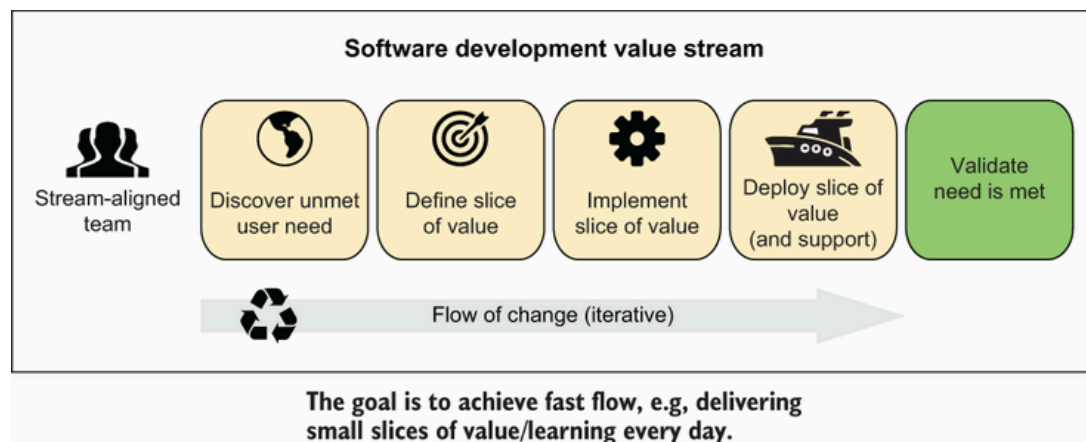


Figure 6.1 The high-level activities in an independent value stream より引用

30/70の例は、実装に着手してからリリースするまでだけを切り出しました。バリューストリームは、さらに広い範囲を扱います。まだ片付いていないユーザーのジョブを見つけ、届けたい結果と確かめ方を決め、実装して届け、ジョブが片付いたか確かめるまでを、ひと続きの仕事として捉えます。

開始条件 ユーザーのジョブが、まだ片付いていない

完了条件 ジョブが片付き、期待した結果を確認できた

出典：Susanne Kaiser, *Architecture for Flow*, Addison-Wesley, 2025, 第6章

実装は、バリューストリームの真ん中にある

ジョブを見つける

ユーザーが困る状況、いま試している手段、達成したいことを、観察や対話から確かめる。

結果と確かめ方を決める

ユーザーの行動や事業の数字がどう変われば価値が届いたと言えるかを、実装前に決める。

作って届ける

設計し、実装してテストする。リリースして、ユーザーが変更を利用できる状態にする。

結果を確かめる

実際に使われ、期待した変化が起きたかを見る。続ける、変える、やめるを判断する。

コードを書く前には判断があり、届けた後には結果の確認がある。

参考：Susanne Kaiser, *Architecture for Flow*, Addison-Wesley, 2025, 第6章



独立したバリューストリームの四つの条件

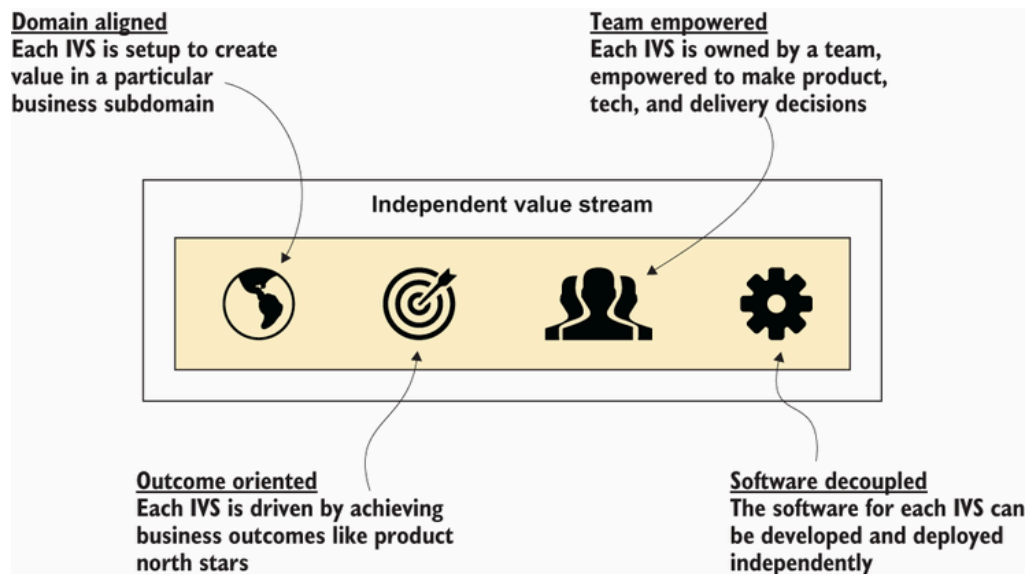


Figure 1.4 Independent value stream より引用

事業の仕事に合わせて担当範囲を決める どのジョブを扱うかが明確

チームが判断できる 製品・技術・リリースを決められる

利用後の変化から作るものを決める 届けた結果まで確かめる

ソフトウェアを分ける 単独で変更して届けられる

ほかのチームと協力していても、日常的な変更を発見から結果の確認まで同じチームで進められるなら、ここでいう**独立**に当たります。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, Manning, 2024, 第1章

四つの条件は、「何を担うか」と「どう進めるか」

事業の仕事に合わせて担当範囲を決める

「注文」「決済」のように、事業の中で一つの役割を担う範囲へ集中する。関係の薄いジョブを同じチームへ集めない。

チームが判断できる

何を作るか、どう実装するか、いつ届けるかをチームで決める。変更のたびに外部の承認を待たない。

利用後の変化から作るものを決める

機能の完成ではなく、利用後に起きてほしい変化を目標にする。届けた後の結果まで同じチームが確かめる。

ソフトウェアを分ける

ほかのシステムを同時に変えなくても、開発・テスト・リリースできる。依存があっても、接続方法やデータ形式を安定させる。

事業の仕事と利用後の変化から「何を引き受けるか」を決める。判断の権限とソフトウェアの分け方で「待たずに終わらせるか」を決める。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, Manning, 2024, 第1章



仕事を引き継ぐところで、待ちが生まれやすい



ここでは、判断や作業を一つの役割から別の役割へ引き継ぐところを**責任の境界**と呼びます。この例では、別の役割の判断が必要になるたびに待ちが発生していました。引き継ぐたびに説明と調整が加わり、次の作業へすぐには進めなくなります。

AIで作業を速くしても、引き継ぐ回数と待つ順番は変わらない。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, Manning, 2024, 第1章 / Susanne Kaiser, *Architecture for Flow*, Addison-Wesley, 2025, 第5章、第6章「依存関係の分析」

依存は、待つ理由で三つに分ける

依存とは、自分たちだけでは次へ進めない状態です。何を待っているかで、対処が変わります。

仕組みへの依存

別のシステムや接続先の変更を待つ
→ 接点を安定させ、別々に変更できるようにする

人の知識への依存

特定の人に聞くまで判断できない
→ 判断の根拠と手順を共有する

順番への依存

先の確認や承認が終わるまで進めない
→ 同時に進めるか、確認条件を明示する

依存をゼロにはしない。待ちが長い依存から、本当に必要かを確認、待ちを減らす方法を決める。

参考：Susanne Kaiser, *Architecture for Flow*, 第6章「依存関係の分析」 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第1章



AIが速くするのは、まず一人の手元

内側のループ

一人が手元でコードを書き、動かし、直す繰り返し。この繰り返しはAIで短くしやすい。

外側のループ

チームがレビュー、統合、検証、説明を重ね、変更を利用できる状態にするまでの繰り返し。

内側のループで作ったPRは、外側のループへ入ります。AIでPRを作る時間が短くなっても、レビューする人数、確認する項目、他チームと合意する順番は、そのままでは変わりません。作業が数日ではなく数時間で終わるようになると、チームが調整するより速く仕事の列が空になり、同じファイルで互いにぶつかり始めます。

内側から外側へ渡す仕事だけが増えると、外側の工程で待ちが増える。

参考：Alfonso Graziano, *AI-Native Software Engineering*, O'Reilly Early Release, 2026年時点の草稿, 「SDDフレームワーク」の章



速くする工程を誤ると、待ちは長くなる

一番遅い工程が、全体の速さを決めます。この工程を**制約**と呼び、まずその工程を改善する考え方が**制約理論**です。内側のループを速めた結果、外側が制約になったとします。外側で一定時間に処理できる仕事の量（処理能力）が変わらなければ、終わっていない仕事（仕掛かり）が積み上がります。

実装が速くなると

たとえば月曜の朝、5つのAIエージェントが、それぞれ500行のPRを作ってきたとします。レビューは、意図と影響範囲を一緒に確かめる作業から、AIが生成した変更を一から読み解く作業へ変わります。レビュアーの時間は増えていません。

レビューが追いつかないと

レビューに使える時間が変わらなければ、1本あたりの待ち時間はむしろ延びます。同僚のPRには追いつけていたレビュアーも、同僚に加えて機械の速さで出荷するエージェントには追いつけません。判断する人の手元には、判断待ちの案件が積み上がります。

実装を速めると、次に遅い工程が全体の速さを決める。この例では、合意と判断。

出典：Eliyahu M. Goldratt（Susanne Kaiser, *Architecture for Flow*, 第6章「制約の管理」での引用）／ Addy Osmani, *Beyond Vibe Coding*, 第10章 ／ Alfonso Graziano, *AI-Native Software Engineering*, 「SDDワークフロー」の章



制約が実装にある現場もある

ただし、待ちが全体の速さを決めるとは限りません。

実装が制約になりやすい段階

1人か2人で、決める人と作る人が同じ。ほかの人への引き継ぎもほとんどありません。ここでは実装が制約になりやすく、AIで実装時間を短くすると、全体も速くなりやすい。

待ちが制約になりやすい段階

役割が分かれ、判断、レビュー、運用のたびに仕事を引き継ぐ現場。待ち時間が全体の大半を占めると、待ちが制約になりやすい。

役割が分かれていても、実装が制約になっている組織はあります。飲食店予約サービスのOpenTableでは、実際に測ると、全体の速さを決めていたのはエンジニアリング工程でした。自分たちの現場で、実装と待ちのどちらが詰まりになっているかも、測って確かめます。次の章では、その測り方を扱います。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第3章



2. 測りやすい数字は、価値が届いた証拠ではない

PR数も、速くなった体感も、AIで伸びやすい。

測りやすい数字はAIで伸びる

PR数、コード変更の記録（コミット）の数、変更行数。どれもダッシュボードに出しやすく、AIを入れると増やししやすい数字です。ただ、これらが表すのは**作った量**です。ある企業では、データベース内で動く処理（ストアドプロシージャ）の作成数が開発者の個人目標になっていました。PR数を個人目標にしても、作った量そのものが目的になる同じ問題が起きます。

アウトプット=作ったもの。アウトカム=使った結果に生じた変化。

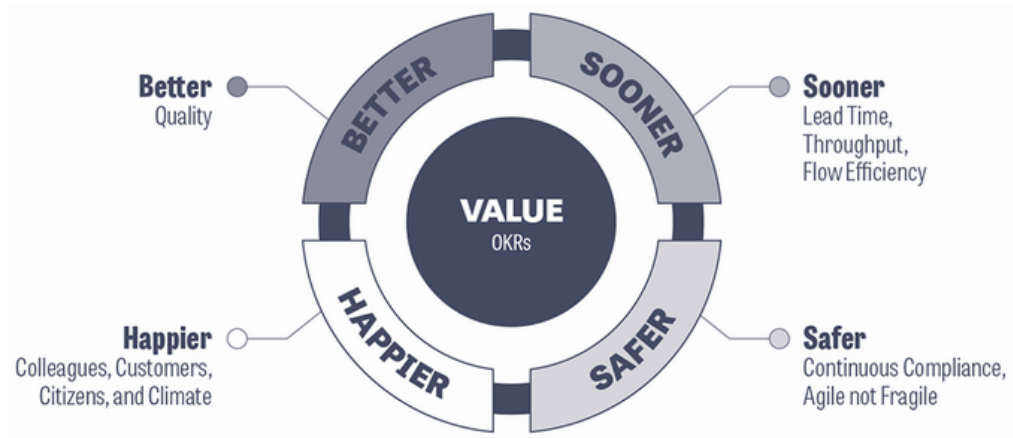
頼まれた機能を作り続け、利用後の変化より機能の量と出す速さだけを見るチームを**フィーチャーファクトリー**と呼びます。機能を作ること自体が目的になり、使われた結果を確かめない状態です。

作った量ではなく、使われた結果を測る。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, Manning, 2024, 第1章、第11章



価値は「速く」だけでは測れない



Better 品質を上げる

Value ユーザーと事業の成果を増やす

Sooner 価値を早く、頻繁に届ける

Safer 変更による失敗と影響を減らす

Happier ユーザーと働く人の満足度を高める

Figure 1.3 Better Value Sooner Safer Happier (Source: Smart et al., Sooner Safer Happier [IT Revolution, 2020]) より引用

AIで早く届けられるようになって、安全性が下がれば、全体として良くなったとは言えない。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, Manning, 2024, 第1章 / Jonathan Smartほか, *Sooner Safer Happier*, 2020

届ける速さを三つに分ける

リードタイム

作業に着手してから、変更を利用可能にするまでの経過時間

スループット

1週間などの一定期間に、利用できる状態まで進んだ変更の件数

流れの効率

リードタイムのうち、実際に手を動かした時間の割合

30/70の例なら、流れの効率は30%。残り70%は待ち時間。

この三つで分かるのは「届ける速さ」です。使われた結果であるアウトカムは、別に確かめます。

参考：Susanne Kaiser, *Architecture for Flow*, 第6章 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第11章



速さと結果は、同じ対象で確かめる

リードタイムやスループットで分かるのは、変更を届ける速さです。価値まで見るには、**同じ期間と対象ユーザー**について、使われ方と結果を並べます。検索条件を追加した例なら、次の三つです。

流れ

着手から利用可能になるまでの時間と、そのうち誰かを待っていた時間

使われ方

対象ユーザーが新しい検索条件を使った割合と、検索を途中でやめた割合

結果

必要な情報へたどり着くまでの時間と、その後に購入へ進んだ割合

届ける時間が短くなっても、使われ方と結果が変わらなければ、価値が届く速さは変わっていない。



作る速さだけを上げると、使われない機能も積み上がる

顧客のジョブよりスケジュールを優先して機能を量産する状態を、Melissa Perriは**ビルドトラップ**と呼びました。作る量だけをAIで増やすと、この罠に入りやすくなります。多くのチームは、新しいものを足すのは得意でも、古いものを消すのは苦手だからです。ある自動車メーカーでは、データベース内で動く処理の約30%がもう使われていませんでした。

機能が1つ増えるたびに

レビューで読む範囲、テストの対象、運用で守る対象、ユーザーが比べる選択肢が増える。

増えた複雑さが、次の変更を遅くする

複雑さは、コードの行数よりも、安全に変更するために把握すべきことの多さとして表れます。古い設定や依存、テストが残っているほど、判断とレビューに時間がかかります。

古いものを減らさず生成だけを増やせば、次の変更は遅くなる。

参考：Melissa Perri, *Escaping the Build Trap* / Nick Tuneほか, *Architecture Modernization*, 第8章



速くなった体感も、測定の代わりにならない

実験

経験豊富なオープンソースソフトウェア（OSS）開発者16人が、日頃から扱っているコード群の246課題に取り組んだ。AIツールを使ったグループは、使わなかったグループより**19%遅かった**。

体感

事前には「24%速くなる」と予想し、終わった後も「20%速くなった」と感じていた。測定と体感が、逆向きだった。

大規模で成熟したコード群に詳しい開発者という条件つきで、ほかの現場へそのまま当てはめることはできません。ただ、少なくともこの条件では、「**速くなった気がする**」だけでは判断できません。体感と、実際に時間を使っているところがずれた例もあります。あるネット銀行では、開発者が「コードを実行可能な形にするビルドが遅い」と訴えました。ところが測ってみると、時間の大半はPRのレビュー待ちでした。

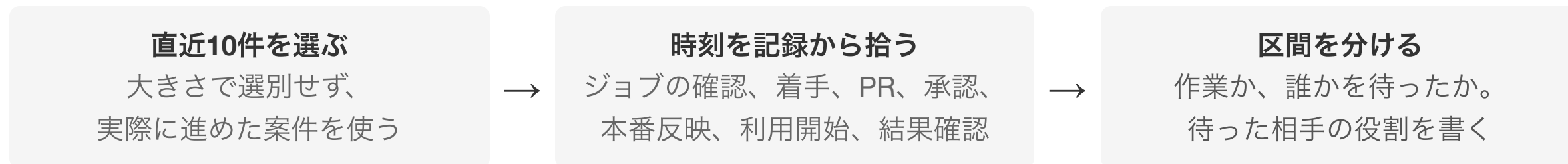
PR数でも体感でもなく、価値が届くまでの流れそのものを測る。

出典：METR, [Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity, 2025](#) / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第13章



価値が届くまでの流れを測る

価値が届くまでの工程を時系列に並べ、作業と待ちを分ける方法をバリューストリームマッピングと呼びます。



最初に見る数字は二つです。全体のうち作業していた時間の割合と、**どの工程で、誰を待った時間が最も長かったか**。ここで見るのは、個人の働き方ではありません。役割の間で仕事をどう渡しているかです。

待ちのすべてが無駄ではありません。本番前の確認のように、安全のため意図して設ける待ちもあります。判断も、担当と判断日を決めて先送りするなら待ちではなく設計です。見直すのは、何のためか説明できない確認、担当が曖昧な引き継ぎ、処理能力を超えて積み上がる待ちです。測れば、必要な待ちと見直す待ちを分けられます。

待ちの合計が大きい工程を、最初の制約候補にする。どの判断や作業を待っていたか確かめる。

参考：Susanne Kaiser, *Architecture for Flow*, 第6章「バリューストリームマッピング」／Jacqui Read, *Communication Patterns*, O'Reilly, 2023, 第11章



3. 役割ごとの前提を対話で確かめる

長い待ちの一部は、情報不足ではなく、役割ごとの前提の違いから生まれる。

役割が違えば、同じ機能でも見え方が変わる

購入前の相談から開発までに関わる役割を例にします。営業、PM（プロダクトマネージャー）、デザイナー、エンジニアは、同じ機能について別の情報を持っています。

営業

顧客との会話
購入の条件

PM

目指す結果
優先順位

デザイナー

利用する場面
操作のつまずき

エンジニア

コードと依存
レビューとテスト

情報が別々の資料や会話に散らばっているだけなら、一か所へ集めれば解決します。難しいのは、同じ情報を見ても、役割が背負う責任によって**何を問題と考えるか**が変わることです。

情報を揃えるだけで、判断の前提まで揃うとは限らない。

参考：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第5章



「違う情報」の奥に、違う解釈がある



宇田川元一『他者と働く』

宇田川元一さんは、物事をどう理解し、何を正しいと考えるか、その**解釈の枠組み**をナラティブと呼びます。立場、経験、背負っている責任が違えば、同じリリース延期にも別の意味が生まれます。

営業 商談の機会を逃す

デザイナー 利用者が途中で迷う

エンジニア 次の変更が難しくなる

SRE 障害対応の負担が増える

どれか一つだけが正しいとは限りません。相手の判断が不合理に見えるときほど、その役割から何が問題に見え、何を守ろうとしているかを確認めます。

「相手が分かっていない」と決める前に、相手からは何が問題に見えているかを確認める。

出典：宇田川元一『他者と働く——「わかりあえなさ」から始める組織論』2019 / [NewsPicks Publishing](#)

仕組みで解ける問題と、対話が要る問題を分ける

『他者と働く』では、既存の知識を当てはめられる**技術的問題**と、関係者が問題の捉え方から見直す**適応課題**を分けます。ソフトウェア開発の待ちには、両方が混ざっています。

技術的問題

原因と目標の認識が揃っており、既存の手順や専門知識を使える。たとえば、ビルド時間をキャッシュで短くする。

適応課題

何を問題とみなすか、何を優先するかが役割ごとに違う。一つの役割だけで解決策を決めても、仕事の進め方は変わりにくい。

承認待ちのすべてが適応課題ではありません。重複した確認は仕組みで減らせます。一方、期日、使いやすさ、変更の難しさ、障害リスクのどれを優先するかで止まっているなら、先に互いの前提を確かめます。

手順の問題は仕組みで減らす。前提の違いから生じる待ちは、対話から始める。

参考：宇田川元一『他者と働く』第1章、第2章 / 「日本の人事部」宇田川元一さんインタビュー



対話は、同意を急ぐことではない

ここでいう対話は、説得や情報共有の言い換えではありません。互いを、指示に従わせる相手ではなく、問題を一緒に扱う相手として関係を作り直すことです。本書では、次の四つを行き来しながら進めます。

準備

分かり合えていないと認め、自分の前提をいったん保留する

観察

相手の言葉、行動、置かれた状況から、何を守ろうとしているかを見る

解釈

相手の立場なら、なぜその判断が合理的なのかを考える

介入

双方が動ける小さな働きかけを試し、反応を見てまた観察する

一度で分かり合うための順番ではありません。相手の反応から見立てを直し、次の働きかけを変える反復です。この発表では、その対話の材料として一枚の図を使います。

図は結論を出すためではなく、違う前提を出し、次に確かめることを決めるために使う。

出典：「日本の人事部」宇田川元一さんインタビュー（準備・観察・解釈・介入）



判断待ちを短くする三つの取り決め

前提を確かめる対話と並行して、判断の依頼そのものにも形を与えます。Jacqui Readは、判断が止まる原因の多くを、誰が決めるのか、何をいつまでに返すのかが曖昧なことに見ています。

決める人を一人にする

最も詳しい人か、最も影響を受ける人が決定オーナー。役職の高い人ではない。

同意ではなくコミットを求める

全員の賛成を待たない。結果に従うことを約束してもらい、意見は記録に残す。

依頼に四つを書く

誰に、何を、いつまでに、返事がないときはどうするか。「なるべく早く」は期限ではない。

会議は、決める権限のある人がいるときだけ同期で開きます。進捗の共有や下書きへの意見は非同期で足ります。10人が1時間集まれば、10時間分の作業が止まります。

判断待ちの多くは、内容ではなく「誰が・いつまでに」が決まっていないことで生まれる。

出典：Jacqui Read, *Communication Patterns*, O'Reilly, 2023, 第12章「意思決定にまつわる神話」、第14章（4つのWはGreene & Sanderson, *Remote Works* より）



対話の材料を、ウォードリーマップに置く

ユーザーが得たい結果、その結果に必要な要素、各要素の成熟度を一枚に並べると、どこへ投資し、どう運用するかを比べられます。この図を**ウォードリーマップ**と呼びます。

ジョブ

誰が、どんな状況で、何を達成したいか

要素同士の依存

待ち時間ではなく、ジョブを満たすために何が何を必要とするか

成熟度

作り方や市場の選択肢が、どれだけ確立しているか

システム構成図は、要素同士の接続を表します。ウォードリーマップでは、そこへ**ユーザーのジョブと成熟度**を加えます。答えを自動で出すための図ではありません。どこへ投資するか、既製サービスと自前運用をどう比べるか、何をやめるかを話すために使います。一枚に全部は描きません。知りたいことが違う図を一枚に混ぜると、誰にも読めなくなります。この図は最も抽象度の高い一枚で、詳細は別の図へつなぎます。

要素だけでなく、「誰のため」と「どの段階」を同時に見る。

参考：Susanne Kaiser, *Architecture for Flow*, 第1章 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第5章 / Jacqui Read, *Communication Patterns*, 第1章「抽象度の混在」



縦は見えやすさ、横は成熟度

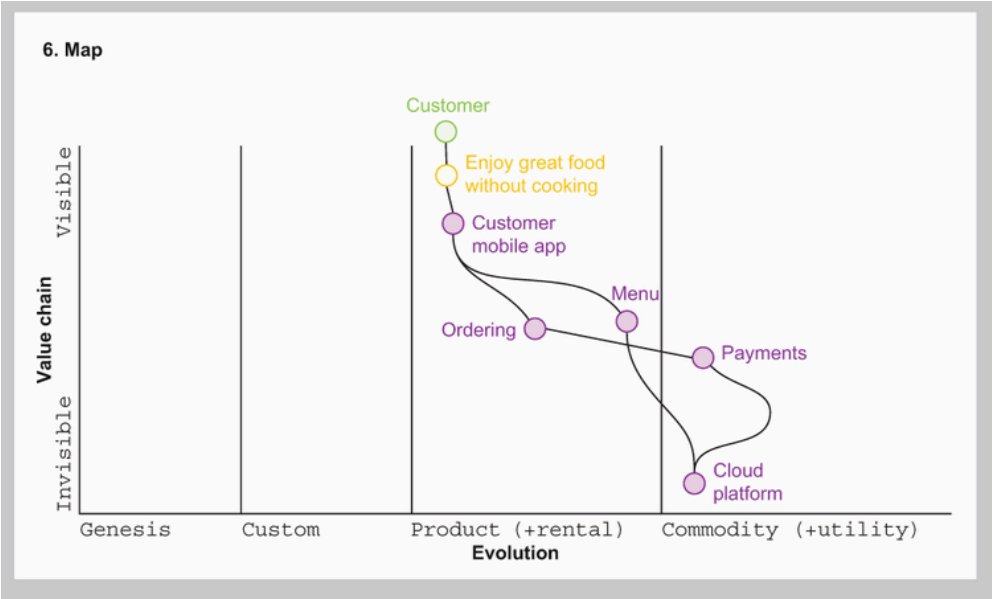


Figure 5.7 Step 6 of the Wardley Mapping Canvas, a Wardley Map より引用

縦はユーザーからの見えやすさ（可視性）です。一番上にユーザーとジョブを置き、そこから下へ、ジョブを満たすために必要な要素を線でつなぎます。線は、上の要素が下の要素を必要とする関係です。上にあるほどユーザーから見えやすく、下にあるほどユーザーからは見えにくい内部の要素です。

横は成熟度です。左から、まだ答えを探している段階、独自に作り込む段階、製品として選べる段階、電気のように誰もが使える共通基盤へ進みます。

横軸は優劣を表しません。置く位置によって、試行錯誤する、独自に作る、製品を選ぶ、共通基盤として扱うなど、適した進め方が変わります。

縦で「なぜ必要か」を確認し、横で「どう扱うか」を考える。

検索の例で、図の読み方を確かめる

先ほどの検索機能を置くと、縦方向にはジョブを支える依存が見え、横方向には要素ごとに適した進め方が見えてきます。

ジョブから依存をたどる

必要な情報へ早くたどり着きたい



検索画面



商品データ・検索エンジン



計算資源・保存領域

成熟度で進め方を変える

どの検索条件が役立つかは、ユーザーと小さく試す。検索エンジンは、既製サービスと自前運用を比べる。計算資源や保存領域は、共通基盤として扱えるかを確かめる。

一つの機能でも、すべてを同じ方法で作る必要はありません。ジョブに近く、まだ答えがない部分には試行錯誤の時間を使います。選択肢が確立した部分は、必要な制御、運用の負荷、総費用で選びます。

どこで試し、どこで既製の選択肢を使うかを、要素ごとに決める。



同じ図に、役割ごとに見ている情報を置く

同じ機能でも、役割によって見ている情報は違います。ここでは、PM、デザイナー、エンジニア、SRE（信頼性と運用を担う役割）が持つ情報を、ワードリーマップに並べます。

PM

誰のどのジョブか
何が変われば価値か

デザイナー

どこで利用者が迷うか
どんな使われ方をするか

エンジニア

どの要素が必要か
変更がどこへ影響するか

SRE

自前で運用すると何が増えるか
障害がどこまで影響するか

誰のジョブを、何が、どの成熟度で支えるかを一枚で確かめる。

参考：Susanne Kaiser, *Architecture for Flow*, 第1章 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第5章



差別化の中心は、横軸の右端にあった

私たちが差別化の中心（コア）だと思って作り込んでいた領域が、あるときクラウド事業者が運用するマネージドサービスと呼ぶ数十行のコードに置き換わりました。

「自分たちで作っている」と「まだ独自に作る必要がある」は別のことです。自分たちが独自だと思っても、市場には既製の選択肢が増え、ユーザーも特別な機能とは見なくなることがあります。ウォードリーマップでは、こうした変化を右への移動として表します。

横軸の右端では、まず既製サービスと比べ、自前運用を続ける理由を問い直します。ただし、必要な制御、性能、障害の切り分け、法令対応、総費用が合わなければ、自前運用へ戻すこともあります。

右端は「買え」ではなく、既製サービスと自前運用を比較し直す合図。

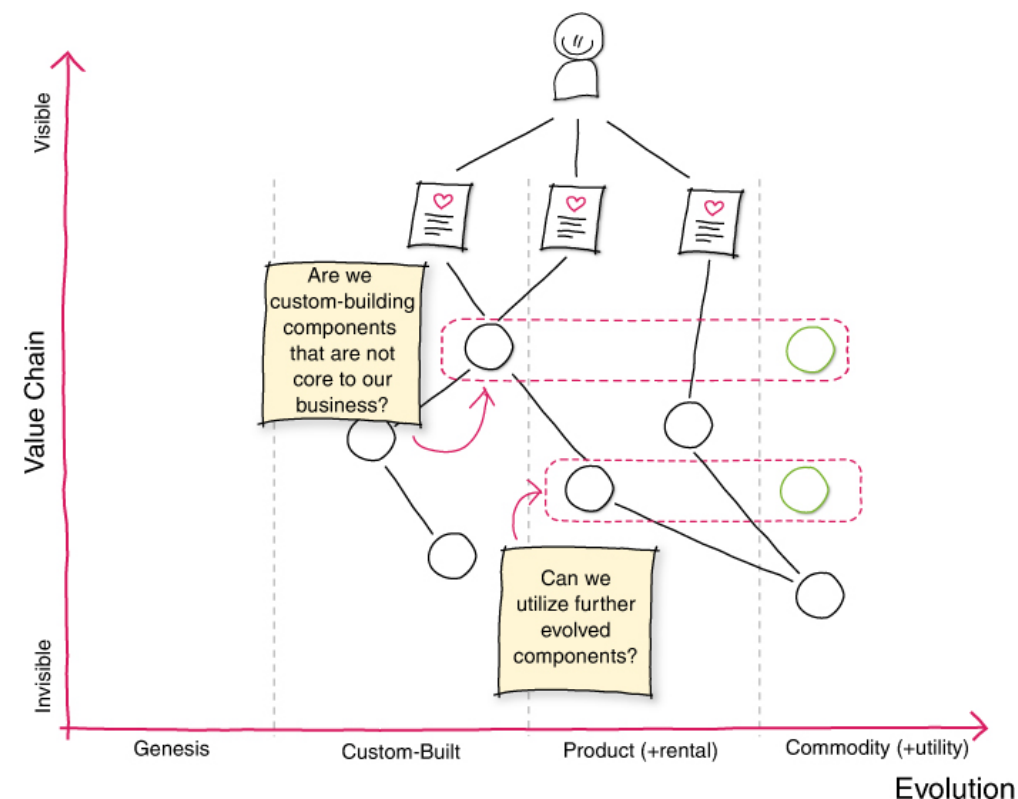


Figure 1.10 Are we investing efficiently? (Architecture for Flow) より引用

出典：Susanne Kaiser, *Architecture for Flow*, 第1章「効率性ギャップ」、第2章 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第5章、第10章

半年計画の刷新は、ジョブにつながっていなかった

半年かける予定だった大規模刷新を、4ヶ月目にロールバックしました。つまり、変更を取り消して元に戻しました。振り返ると、「誰のどのジョブを片付けるのか」を問わないまま、**作れるものを作っていた**。

ワードリーマップで見ると

ユーザーのジョブとの関係を説明できない要素は、どれだけ大きくても価値につながっていない。刷新の対象はシステムの内部で、ジョブとの関係は確かめていなかった。

バリューストリームで見ると

ロールバックで、4ヶ月分の変更は誰のジョブも片付けずに消えた。作れることは分かりました。しかし、作るべきだったかは最後まで確かめられませんでした。

4ヶ月分を取り消すとなると、やめる判断そのものが重くなります。最初の判断が間違いだったように見えることも、決断を遅らせます。判断が疑われるまでの時間が長いほど、直す費用は高くなる。やめる判断を早くする方法は、4章で扱います。

ユーザーのジョブとの関係を説明できない計画は、始める前に問い直せたはず。



図に並べると、何を話し合うかが揃う

PM・デザイナー・エンジニアに、ユーザーと直接話す営業やCS（カスタマーサクセス）も加わって30分だけ集まり、ユーザーのジョブ、必要な要素、依存をざっくり並べます。最初から正確でなくても、見ている情報の違いが分かれば話し合いを始められます。

ジョブとの関係を説明できない要素

今回の範囲外。捨てずに、別のジョブとの関係を確かめ直す

横軸の右端にある要素

既製サービスと自前運用を比較。必要な制御と総費用で決める

依存の線が集まる要素

複数の機能や仕組みが共通して必要とする要素。2章で測った待ち時間と見比べ、担当するチームを決める

図の目的は、関係者全員が見ている情報を揃え、投資を決める理由をはっきりさせることです。最初から現状の細部まで正確に描く必要はありません。まず、合意にかかる時間を短くします。次に、**ジョブの発見から結果の確認までを、どのチームが担当するか**を見直し、チーム間の引き継ぎそのものを減らします。

図で見ている情報を揃える。計測で、減らす待ちを選ぶ。

参考：Susanne Kaiser, *Architecture for Flow*, 第5章「フロー最適化の要件」、第8章 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第10章



4. ジョブから「作る・やめる」を判断する

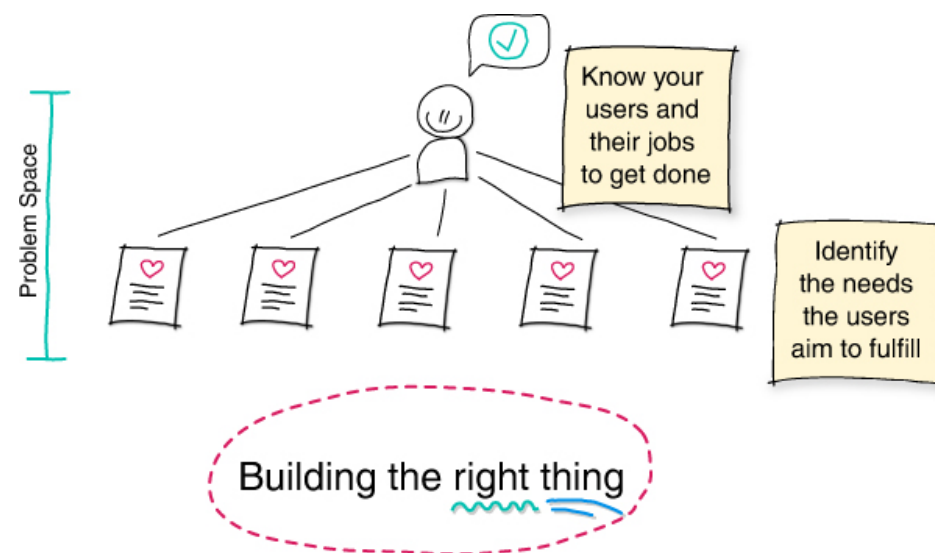
作れるものが増えても、ユーザーのジョブは増えない。

ジョブは、解決策が変わっても残る

ジョブ理論では、ユーザーは目的を果たすためにプロダクトを「雇う」と考えます。機能が揃っていてもジョブが片付かなければ、別の手段へ乗り換えます。

状況とジョブが同じなら、**解決策を変えてもジョブそのものは変わりません**。AIで作れるものは増え続けますが、ユーザーのジョブは、私たちの実装能力に合わせて増えません。

作れるものがいくら増えても、ユーザーのジョブは増えない。



"People hire products or services to get a job done."

Clayton Christensen

Figure 1.1 Starting from the user perspective to build the right thing (Architecture for Flow)
より引用

出典：Christensen Institute, Jobs to Be Done Theory / Susanne Kaiser, Architecture for Flow, 第1章

作るべきかは、四つの問いで決める

作るものとジョブの関係を説明できるか

手段を消し、「どんな状況で、何を達成したいのか。なぜそれが大切か」を一文で書く

最近、実際に困った場面があったか

抽象的な賛否より、直近の行動を見る。そのとき何を試し、何が決め手だったか

待ち時間の合計が最も大きい工程を改善するか

2章で見つけた待ち時間を減らさない機能は、作っても待ちの列に並ぶだけ

既存の手段や小さな実験で、先に確かめられないか

まず既製サービスと比べる。必要な制御や総費用が合わなければ自前で持つ。不確かなら小さく試す

答えられない問いがあれば、実装を始める前に、ユーザーの行動や実際の待ち時間を確かめる。

機能を消す方が、作るより難しい

機能を加えるときは、期待する新しい動作を満たすことに確認の範囲を絞りやすい。一方、消すときは、**今ある大切な動作を失わないこと**を確かめなければなりません。その判断材料は、コードだけには残っていません。

利用実態と重要性を判断しにくい

短期間の利用記録だけでは、誰が、どんな状況で使っているかを捉えにくい。利用回数が少なくても、特定の顧客の業務や障害時の復旧に欠かせないことがあります。

依存が見えにくい

設定、データ、バッチ、帳票、運用手順、他チームの仕組みが、明示されないまま機能に依存していることがあります。

利用記録だけで決めず、営業やCSには顧客への影響を、エンジニアには仕組みの依存を、SREや運用担当には障害時の使われ方を確かめます。新しい利用を止め、関係者へ知らせ、元に戻せる状態で一度止めてから削除します。

「使われていないように見える」だけでは消せない。影響を確かめ、段階を踏んで止める。

参考：Susanne Kaiser, *Architecture for Flow*, 第5章 / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 第16章



C4モデルで削除の影響を確かめる

C4モデルは、ソフトウェアの構造を、システム全体からコードまで段階を分けて表す方法です。削除の判断では、四段階すべてを描くのではなく、まず上の二つで影響範囲を確かめます。

システムコンテキスト図

対象システムを一つの箱として、その周りに利用者と外部システムを置きます。機能を消したとき、誰の仕事と、どの外部連携に影響するかを確かめます。

コンテナ図

システム内のアプリケーションとデータストア、その通信を表します。ここでいうコンテナは Docker コンテナではなく、実行するアプリケーションかデータストアを指します。

複雑なところだけ補う 機能が動く順序は動的図、本番で動く場所や冗長化・代替経路は配置図で確かめます。コンポーネント図やコード図まで必要かは、削除による失敗の大きさで決めます。

必要な粒度まで詳しくする。すべての図を作ることを目的にしない。

出典： [C4 model, Diagrams](#) / Simon Brown, *The C4 Model*, 2026, 第2〜4・7〜8章



一つの記録だけでは、削除を判断できない

C4図が示すのは、設計判断の結果としての構造や動作です。なぜその設計を選んだのか、いま誰が必要としているのかは、別の情報とつないで確かめます。

利用実態と重要性

利用記録に加え、営業・CS・ユーザーへの確認から、誰がどんな状況で使い、その機能を止めると何に困るかを見る。

現在の構造と動作

C4図をコード、設定、データ、稼働環境と照合し、削除対象から影響するシステムと代替経路を確認する。

判断した背景

検討中はDesign Doc（設計案と検討内容をまとめる文書）に選択肢と影響を書く。決めた後はADR（設計判断の記録）に、選んだ理由、受け入れた不利益、見直す条件を残す。

図が古ければ影響を見落とし、ADRだけでは現在の利用を見誤ります。すべてを一つに詰め込まず、C4図、Design Doc、ADR、利用記録を相互にリンクし、判断日には同じ機能について見直します。

ログだけでは重要性が分からない。図だけでは理由が分からない。ADRだけでは現状が分からない。

参考：Simon Brown, *The C4 Model*, O'Reilly, 2026, 第12章 / Jacqui Read, *Communication Patterns*, O'Reilly, 2023, 第12章



いつやめるかは、作る前に決める

機能を消すのが難しいからこそ、削除の判断を後回しにしません。先ほどの半年計画では、4ヶ月目まで中止を判断できませんでした。やめる条件と判断日を、機能を作る前に書いておけば、継続か中止かをもっと早く決められます。「2ヶ月の調査が終わる7月16日に、続けるかを決める」のように。

ジョブが別の手段で片付いている
マネージドサービスや既存機能で足りる。置き換える

想定したジョブには使われていない
別のジョブを確かめるか、機能を消す

維持が流れを遅くしている
変更のたびにレビュー範囲と障害を増やす。消す

判断日には、利用記録、問い合わせ、依存先、障害時の代替手段を見直します。条件を満たしていなければ、同じ形で続けるのではなく、縮小・停止・置き換えを選びます。使った費用の大きさは、続ける理由になりません。

「この条件になったらやめる」と「いつ判断するか」を、
PRの説明、ADR（設計判断の記録）、Design Doc（設計案と検討内容をまとめる文書）のいずれかに記載する。

参考：Kaiser, *Architecture for Flow*, 第5章 / Tune・Perrin, *Architecture Modernization*, 第16章 / Read, *Communication Patterns*, 第11章



検索機能の実装前に決めること

作ると決めたら、目標、守る条件、確認方法、対象外を実装前に書きます。この四つを**実装前の仕様**と呼びます。仕様駆動開発（SDD）の受け入れ基準、非目標、完了チェックに当たります。

目標

対象ユーザーが必要な情報へ早くたどり着き、購入を判断できる

守る条件

検索結果の正しさ、応答時間、閲覧権限を、変更前より悪化させない

確認方法

テストとログに加え、検索を終えるまでの時間と購入へ進んだ割合を見る

対象外とやめる条件

推薦機能は含めない。判断日までに利用や結果が変わらなければ、続け方を見直す

AIに実装を頼む前に、何を変え、何を守り、どう確かめるかをチームで決める。

仕様のレビューは、何を作るかを職能を越えて揃える場です。仕様の失敗の多くは、ある役割が知っていた制約を別の役割が知らなかったことから起きます。その会話は数分。コードレビューで見つければ数時間、本番なら、もっと高くつきます。



AIは作れる。人が作るべきかを決める

実装前の仕様をAIへ渡すと、目標と守る条件に沿った案を出し、確認を繰り返しやすいです。生成が速いほど、曖昧な指示から異なる実装が次々に生まれます。曖昧さの影響範囲は、作る速さに比例して広がる。AI支援開発で最も多い無駄は、悪いコードではなく、間違った問題を解くコードです。エージェントは実装の判断はできても、意図の判断はできません。そして受け入れ基準が書けないなら、まだ探索の段階です。その機能を作るべきかは、仕様だけでは決まりません。

AIに任せる

計画と実装の候補
反復とテスト・ログの収集

人が決める

ジョブと守る条件
確認方法とやめる条件

AIに任せる範囲は、検証できるところまで。作るべきかは、チームが決める。

参考：Alfonso Graziano, *AI-Native Software Engineering*, 「仕様駆動開発」の章 / Addy Osmani, *Beyond Vibe Coding*, 第3章・第4章



まとめ 実装を速めても待ちは残る

AI支援で、動くコードを書く時間は短くなり、個人が作れる量も増えます。ただし、ジョブの発見から結果の確認までには、実装以外の時間があります。

実装

設計・実装・テスト。AI支援で短くしやすく、PR数や生成量にも表れやすい。

待ち

優先順位の判断、レビュー、他の役割との合意。仕事の進め方を変えなければ残る。

結果の確認

本番で使われ、狙った変化が起きたかを確認する。リリースしただけでは分からない。

**30/70の例では、実作業を半分にしても全体は15%しか縮まらない。
速くなった実感と届いた価値が噛み合わないのは、実装の外にある待ちが残るから。**



まとめ 作るべきかは流れとジョブで決める

「作るべきか」を判断するには、次の三つを一緒に見ます。

流れを測る

直近10件を作業と待ちに分け、どの工程で、誰を待った時間が最も長かったかを確認める。

前提を確認める

役割ごとに何が問題に見え、何を守ろうとしているかを対話する。その材料を一枚の図に並べる。

ジョブから決める

最近困った場面と、何が変われば片付いたと言えるかを確認める。既存の手段で足りるなら作らない。

AIは「どう作るか」の候補を増やす。

「何を作るか」「いつやめるか」は、計測とジョブの理解をもとにチームが決める。



参考資料

フローとプロダクト

- 宇田川元一『他者と働く——「わかりあえなさ」から始める組織論』2019
- Jacqui Read, *Communication Patterns*, O'Reilly, 2023（邦訳『開発者とアーキテクトのためのコミュニケーションガイド』オライリー・ジャパン, 2025）
- Simon Brown, *The C4 Model*, O'Reilly, 2026
- Susanne Kaiser, *Architecture for Flow*, Addison-Wesley, 2025
- Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, 2024
- Clayton M. Christensenほか, *Competing Against Luck*, 2016 / [Jobs to Be Done Theory](#)
- Melissa Perri, *Escaping the Build Trap*, 2018 / Rob Fitzpatrick, *The Mom Test*, 2013
- Jonathan Smartほか, *Sooner Safer Happier*, 2020 / Eliyahu M. Goldratt, Jeff Cox, *The Goal*, 2004

AI時代の開発

- Addy Osmani, *Beyond Vibe Coding*, 2025
- Alfonso Graziano, *AI-Native Software Engineering*, O'Reilly Early Release（2026年時点の草稿、刊行予定2027年）
- [DORA, State of AI-assisted Software Development 2025（v. 2025.2）](#)
- [DORA, ROI of AI-assisted Software Development 2026（v. 2026.1）](#)
- [METR, Early-2025 AI on Developer Productivity](#)



3>SHAKE

AIは作れる。

作るべきかを決めるのが私たちの仕事。

職能の壁を越えて価値のフローを設計する

ありがとうございました

@nwiizo